Onyx User Manual

Onyx runs as a set of Docker Compose services and connects to an LLM model served separately by aiDAPTIV middleware. This manual uses our repository, which already includes the aiDAPTIV KV-cache settings integrated to onyx.

Part 1: Before You Begin

Setting up Docker Desktop

If you don't already have Docker installed, get Docker Desktop first , everything else in this manual depends on it.

1. Download Docker Desktop from the official site:

https://www.docker.com/products/docker-desktop/

2. Run the installer and accept the defaults. On Windows, make sure the WSL2 option is selected if prompted — this is Docker's recommended backend.
3. Restart your machine if the installer asks you to.
4. Open Docker Desktop and wait until it says “Engine running” (a green status in the bottom-left corner) before continuing.
5. Confirm it from a terminal:

   ➔ Open and run this command in a seperate new terminal



Cloning our aiDAPTIV-Onyx Repository

Use our aiDAPTIV-Integration-Onyx repository. The miphi-dev branch already has the aiDAPTIV cache settings built into docker-compose.yml, so you can skip straight to docker compose up -d without editing any files yourself:

https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-onyx.git

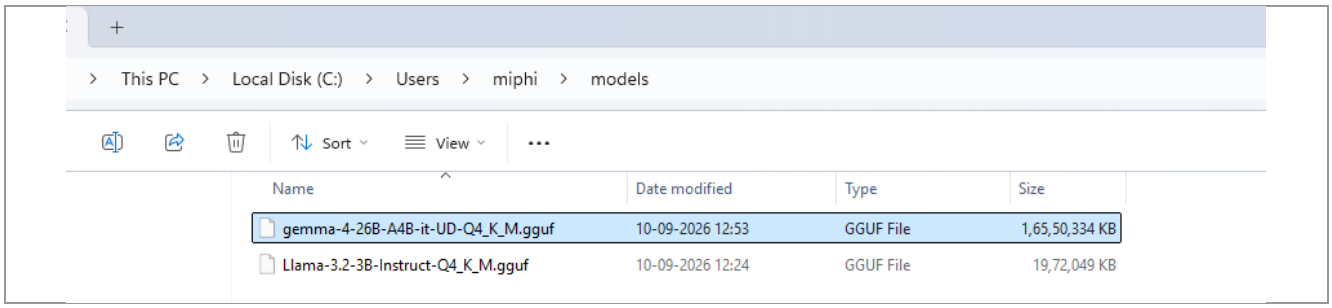
```
git clone https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-Integration-onyx.git onyx
cd onyx/deployment/docker_compose
```

From here, follow Onyx's own official Docker deployment guide for everything else (the .env setup, docker compose up -d, and first-time browser setup):

https://docs.onyx.app/deployment/local/docker

The rest of these prerequisites should also be in place before you start Onyx:

6. Your LLM model file (a .gguf file) should be saved on the disk that your aiDAPTIV middleware can reach.
7. The aiDAPTIV middleware is installed as per its own setup guide, and the aiDAPTIV NVMe cache path it uses is configured with the middleware.



Example model folder — the .gguf file(s) your llama-server will load.

8. Start the LLM model server (via the aiDAPTIV middleware) pointed at that models folder, and confirm it reports a listening address, e.g. http://127.0.0.1:8080. Onyx will connect to this address in Part 3.

> **ℹ NOTE**
> Onyx itself does not load the model. It talks to whatever OpenAI-compatible endpoint your aiDAPTIV Middleware exposes. Get that server running and confirmed first.

Part 2: What's Already Configured For You

This section explains the configuration setup of aiDAPTIV

1. Inside api_server's environment: section, the additional parameters required to offload and reuse the KVCache is already present.

```
environment:
  # Auth Settings
  - 'LITELLM_EXTRA_BODY={"cache_prompt": true,"offload_folder_name": "test_2"}'
  - AUTH_TYPE=${AUTH_TYPE:-basic}
```

```yaml
name: onyx

services:
  api_server:
    image: ${ONYX_BACKEND_IMAGE:-onyxdotapp/onyx-backend:${IMAGE_TAG:-latest}}
    build:
      context: ../../backend
      dockerfile: Dockerfile
      # The shipped image; the Dockerfile's default (last) stage is the dev variant
      # with debugging tools (published with a -dev tag suffix).
      target: runtime
    command: >
      /bin/sh -c "alembic upgrade head &&
      echo \"Starting Onyx Api Server\" &&
      uvicorn onyx.main:app --host 0.0.0.0 --port 8080"
    # Check env.template and copy to .env for env vars
    env_file:
      - path: .env
        required: false
    depends_on:
      relational_db:
        condition: service_started
      opensearch:
        condition: service_started
      cache:
        condition: service_started
      inference_model_server:
        condition: service_started
      minio:
        condition: service_started
        required: false
    restart: unless-stopped
    # DEV: To expose ports, either:
    # 1. Use docker-compose.dev.yml: docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --wait
    # 2. Uncomment the ports below
    # ports:
    #   - "8080:8080"
    environment:
      # Auth Settings

  # Auth Settings
    - 'LITELLM_EXTRA_BODY={"cache_prompt": true,"offload_folder_name": "test_2"}'
    - AUTH_TYPE=${AUTH_TYPE:-basic}
    - FILE_STORE_BACKEND=${FILE_STORE_BACKEND:-s3}
    - POSTGRES_HOST=${POSTGRES_HOST:-relational_db}
    - OPENSEARCH_HOST=${OPENSEARCH_HOST:-opensearch}
    - OPENSEARCH_ADMIN_PASSWORD=${OPENSEARCH_ADMIN_PASSWORD:-StrongPassword123!}
    - REDIS_HOST=${REDIS_HOST:-cache}
```

The api_server service block in docker-compose.yml — note the environment: section near the bottom, already containing the aiDAPTIV cache line.

What the two keys do:

Key	What it does
`cache_prompt`	Tells the llama-server to save the KV cache generated during prefill instead of discarding it once the reply is sent, and to check for a matching cache before recomputing on the next request.
`offload_folder_name`	The subfolder name, under your aiDAPTIV NVMe cache root, where this model's KV-cache files are written (in this example, “test_2”). You will see this exact name again in Part 5 when you go looking for the cache files on disk.

2. If you ever need a different offload_folder_name (e.g. to keep separate caches per model or benchmark), edit that one line and recreate just that service — no need to touch anything else:

```
docker compose up -d --force-recreate api_server
```

ℹ NOTE

Because this fork already ships with the setting in place, most users can skip straight from Part 1's docker compose up -d to Part 3 — this Part is here for reference and verification, not as a required editing step.

Troubleshooting Step

If onyx-nginx-1 container fails to start run the below commands in your powershell to fix it

#First Command :

```
$path = "C:\Users\aidap\onyx\deployment\data\nginx\run-nginx.sh"
$bytes = [System.IO.File]::ReadAllBytes($path)
$text = [System.Text.Encoding]::UTF8.GetString($bytes)
$text = $text -replace "`r`n", "`n"
$text = $text -replace "`r", "`n"
[System.IO.File]::WriteAllText(
  $path,
  $text,
  (New-Object System.Text.UTF8Encoding($false))
)
```

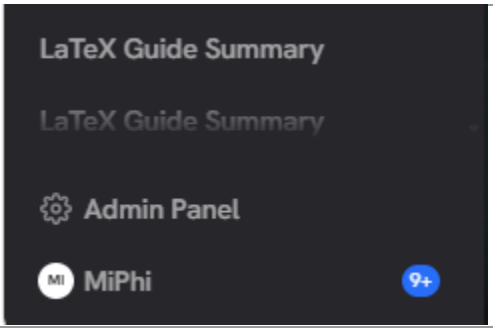
#Second Command :

```
$bytes = [System.IO.File]::ReadAllBytes("C:\Users\aidap\onyx\deployment\data\nginx\run-nginx.sh")
(($bytes | Where-Object { $_ -eq 13 }).Count)
```

Part 3: Connecting Onyx to Your LLM Model

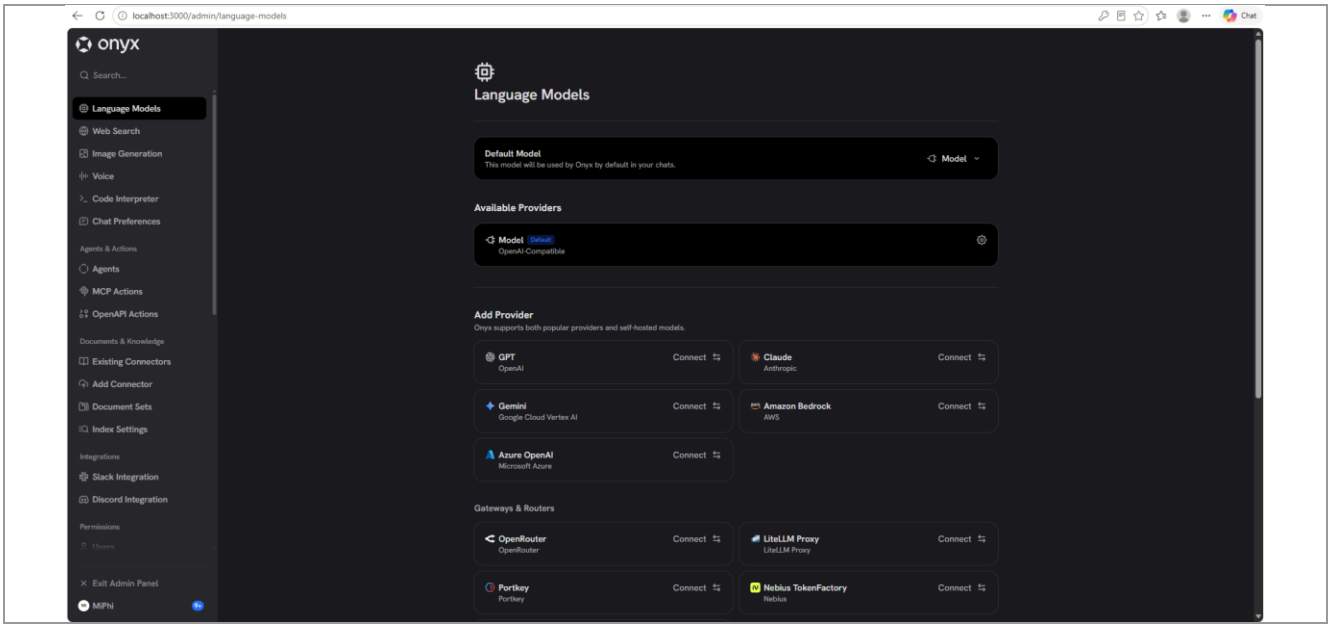
With the model server running (Part 1) and the cache setting in place (Part 2), point Onyx at the model.

1. Go to the URL http://localhost:3000 and Open Onyx in your browser and, from the left sidebar, click Admin Panel. The first user created in Onyx has the Admin Privilege.



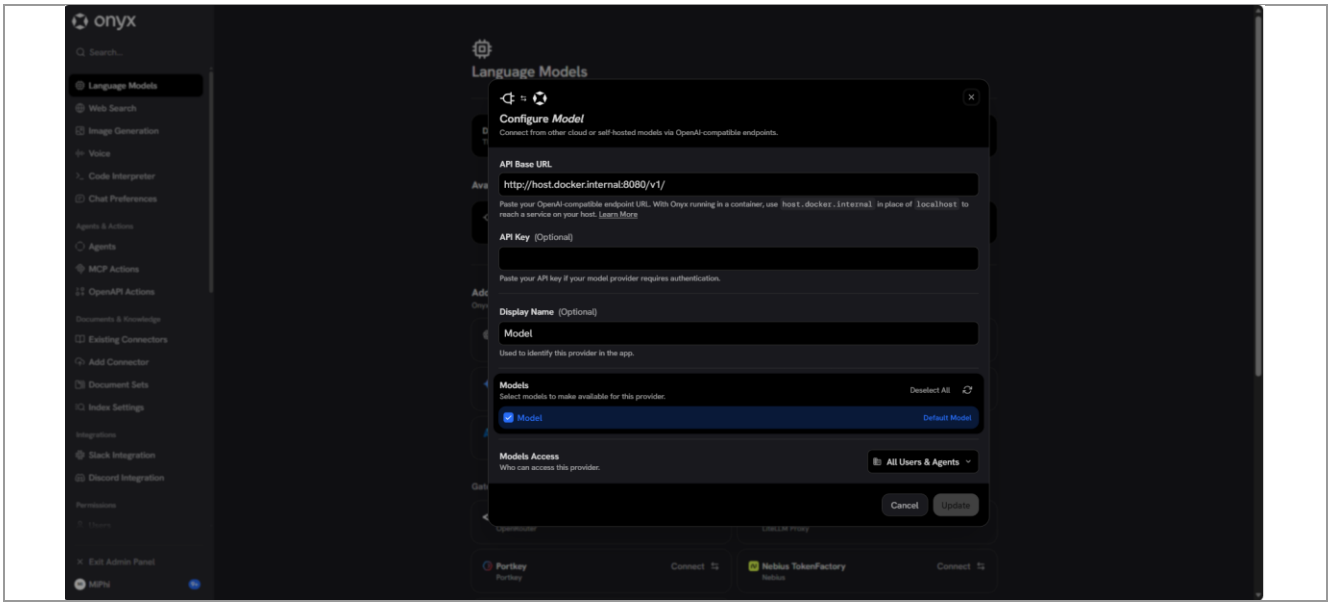
Admin Panel entry point at the bottom of the left sidebar.

2. In the Admin Panel, click Language Models.



Admin → Language Models. The default 'Model' provider (OpenAI-Compatible) is listed under Available Providers — click its gear icon to configure it, or Connect under Add Provider to add a new one.

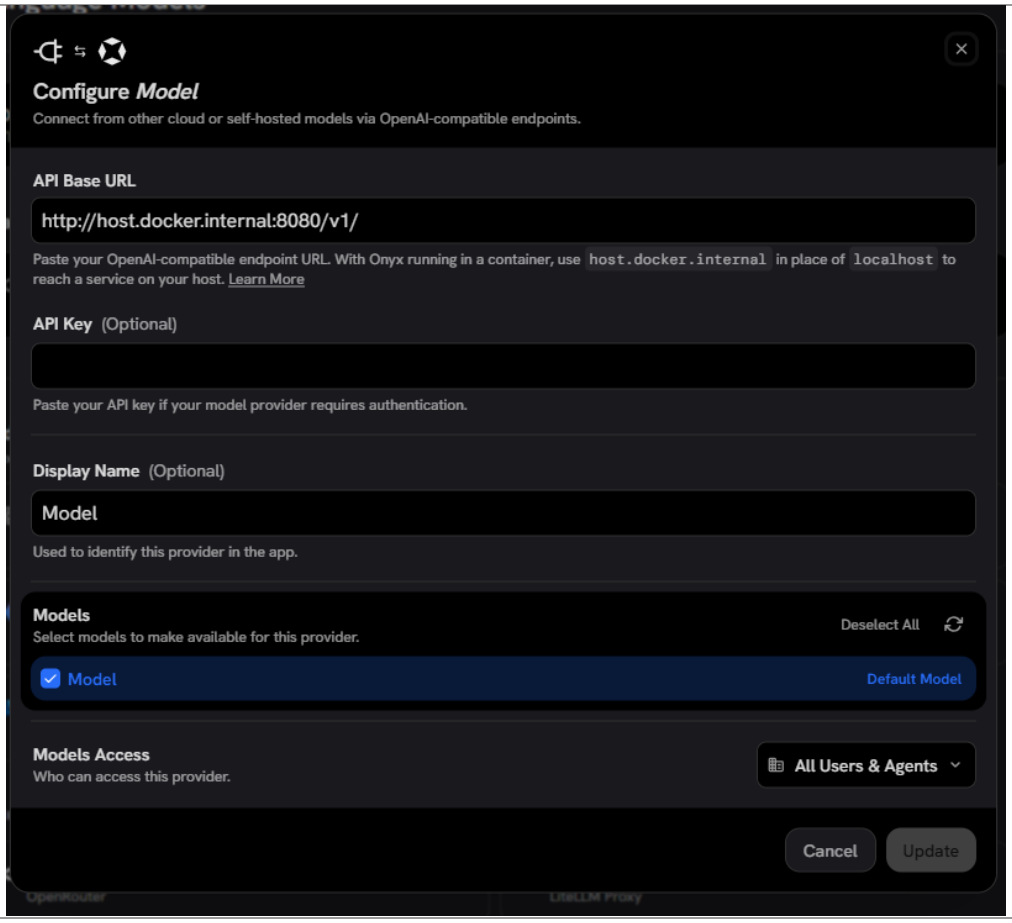
3. In the Configure Model dialog, fill in the fields as follows:



Configure Model dialog.

Field	What to enter
API Base URL	http://host.docker.internal:8080/v1/ — use host.docker.internal, not localhost or 127.0.0.1. Onyx's containers run on their own network and cannot see your host machine's localhost; Docker's host.docker.internal name resolves to it instead.
API Key	Leave blank unless your llama-server is configured to require one.

Field	What to enter
Display Name	Optional — a label to identify this provider in the app, e.g. “Model.”
Models	Check the box next to your model so it is selectable, and confirm it is marked as the Default Model.
Models Access	Choose who can use it — e.g. All Users & Agents.

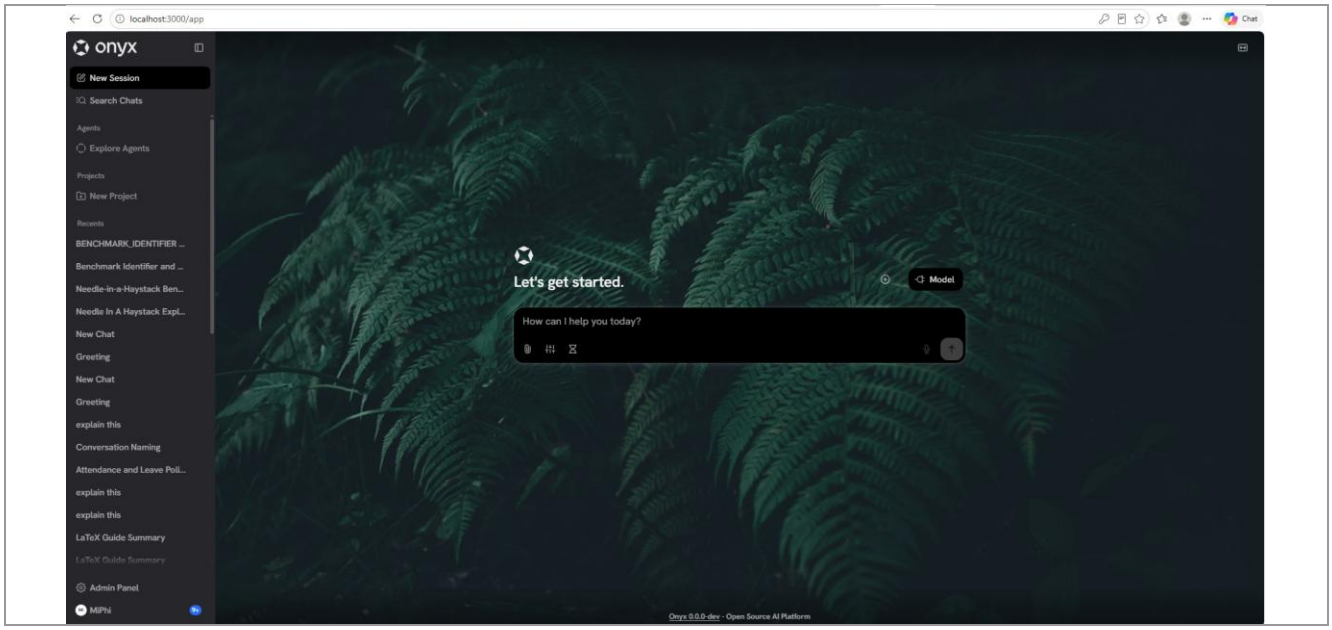


Configure Model dialog, close-up — API Base URL, Models list with the model checked as Default Model, and Models Access.

4. Click Update to save.

🎯 Part 4: Getting Started

1. Open http://localhost:3000/app in your browser.



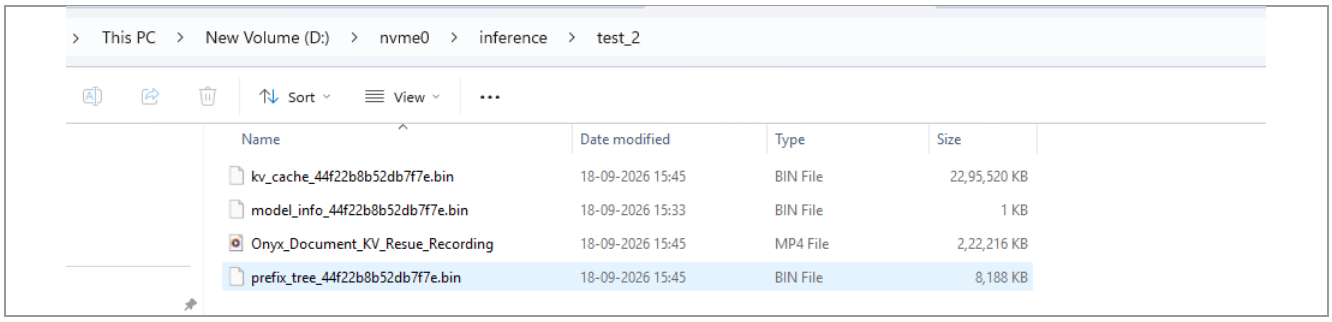
The Onyx 'Let's get started' landing screen.

2. Confirm the Model selector (top of the message box) shows the model you connected in Part 3.
3. Type your prompt — or attach a file first, then type your question — and send it.

☑ Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working

Check the cache folder on disk

After sending at least one prompt, browse to your aiDAPTIV NVMe cache root plus the offload_folder_name you set in Part 2 (in this example, test_2). New files confirm the KV cache is actually being written to the aiDAPTIV SSD, not just held in VRAM.



Example cache folder D:\nvme0\inference\test_2, showing kv_cache_*.bin, model_info_*.bin, and prefix_tree_*.bin files written by the llama-server.

confirm a cache hit

Ask a prompt for a first time, The KVCache of the prompt will be saved in aiDAPTIV.Send the same (or an overlapping) prompt again and check the log for the next request:

```
0.18.674.803 I srv          init: init: chat template, thinking = 1
0.18.674.826 I srv  llama_server: model loaded
0.18.674.829 I srv  llama_server: server is listening on http://127.0.0.1:8080
0.18.674.841 I srv  update_slots: all slots are idle
0.59.450.948 I srv  params_from_: Chat format: peg-gemma4
0.59.490.387 I slot get_availabl: id  3 | task -1 | selected slot by LRU, t_last = -1
0.59.490.564 I slot launch_slot_: id  3 | task 0 | processing task, is_child = 0
0.59.490.705 I slot update_slots: id  3 | task 0 | Hit token cnt (GPU): 0
0.59.848.849 I slot update_slots: id  3 | task 0 | Hit token cnt (DRAM): 0
0.59.848.862 I slot update_slots: id  3 | task 0 | Hit token cnt (CACHE): 10976
[MDW][Info][Runtime] Save 0 tokens to folder 'test_2'
```

Second-run log: Hit token cnt (CACHE): 10976, and 0 new tokens saved.

A large, non-zero Hit token cnt (CACHE) value — 10,976 tokens in this example — is direct proof that those tokens were retrieved from the aiDAPTIV NVMe cache instead of being recomputed on the GPU. The closing line, “Save 0 tokens to folder 'test_2'”, confirms nothing new needed to be cached because it was already there. This lower recompute workload is what produces the faster Time To First Token you will see on repeated or overlapping prompts.

> **ℹ NOTE**
>
> If your second run still shows Hit token cnt (CACHE): 0, double-check that: the LITELLM_EXTRA_BODY line from Part 2 was added correctly and the containers were recreated (docker compose down && up -d) afterward; the prompt overlaps meaningfully with the first one; and the offload_folder_name folder has free space on the aiDAPTIV SSD.

Part 6: Benchmark Comparison — TTFT, TPS & Cache Hit.

Run 2 — Cache Hit (KV Reuse)

From our benchmarking test. A previous run had already built up a 12,752-token cache for this prompt; this run reused it:

```
0.39.096.286 I slot get_availabl: id  3 | task -1 | selected slot by LRU, t_last = -1
0.39.096.481 I slot launch_slot_: id  3 | task 0 | processing task, is_child = 0
0.39.096.600 I slot update_slots: id  3 | task 0 | Hit token cnt (GPU): 0
0.39.441.402 I slot update_slots: id  3 | task 0 | Hit token cnt (DRAM): 0
0.39.441.416 I slot update_slots: id  3 | task 0 | Hit token cnt (CACHE): 12752    HIT CACHE
[MDW][Info][Runtime] Save 0 tokens to folder 'benchmark_13k'
0.39.766.394 I slot update_slots: id  3 | task 0 | saved checkpoint to Cache Storage (n_tokens = 12752)
0.41.636.826 I reasoning-budget: activated, budget=2147483647 tokens
0.53.284.748 I slot print_timing: id  3 | task 0 | n_decoded =    100, tg =   8.49 t/s
0.56.344.656 I slot print_timing: id  3 | task 0 | n_decoded =    127, tg =   8.56 t/s
0.57.563.720 I reasoning-budget: deactivated (natural end)
0.59.377.637 I slot update_slots: id  3 | task 0 | Multi-token MoE expert hit rate (GPU): 72.44%
0.59.377.646 I slot update_slots: id  3 | task 0 | Multi-token MoE expert hit rate (DRAM):  0.00%
0.59.377.648 I slot update_slots: id  3 | task 0 | Single-token MoE expert hit rate (GPU): 96.17%
0.59.377.648 I slot update_slots: id  3 | task 0 | Single-token MoE expert hit rate (DRAM):  0.00%
[MDW][Info][Runtime] Save 184 tokens to folder 'benchmark_13k'
0.59.762.249 I slot print_timing: id  3 | task 0 | prompt eval time =    2413.58 ms /    32 tokens (   75.42 ms per token,
13.26 tokens per second)                                                  TTFT
0.59.762.251 I slot print_timing: id  3 | task 0 |        eval time =   17866.91 ms /   155 tokens (  115.27 ms per token,
8.68 tokens per second)  TPS
0.59.762.253 I slot print_timing: id  3 | task 0 |       total time =   20280.48 ms /   187 tokens
```

Real log — benchmark_13k, cache-hit run. Hit Cache: 12,752 tokens. TTFT (prompt eval time): 2,413.58 ms. TPS (eval/decode speed): 8.68 tokens/sec.

Metric	Run 2 — Warm (cache hit)
Hit token cnt (CACHE)	12,752 tokens
TTFT (prompt eval time)	2,413.58 ms (≈ 2.41 s)
TPS (decode / eval speed)	8.68 tokens/sec